

Secure Software Development (SSD) Process

Company-wide, technology-agnostic process for secure software delivery

Table of content

1 Purpose, Objectives, Scope and Applicability 4
1.1 Purpose and objectives 4
1.2 Scope and applicability 5
1.3 Exclusions 5

2 Governance, Segregation of Duties, Roles and Responsibilities 6
2.1 Governance 6
2.2 Roles and responsibilities 7
2.2.1 Standard SDLC roles 7
2.2.2 Agentic development roles 8
2.3 Mandatory Segregation of Duties 8
2.4 RACI matrix 9

3 Architecture of an Agentic Development Pipeline 10
3.1 Mandatory Secure Sandboxing Requirements 10
3.2 Deterministic Guardrails: Human-in-the-Loop (HITL) Triggers 11
3.3 Agent Prompt & Tooling Governance 11

4 SSD Lifecycle Process Overview, Evidence Model and Process Rules 12
4.1 Process overview 12
4.2 SSD process rules 13
4.2.1 Plan security from the start 13
4.2.2 Define and trace security requirements 13
4.2.3 Analyse threats, risks and design controls 13
4.2.4 Apply secure design principles 13
4.2.5 Select and follow implementation standards 13
4.2.6 Build, review and validate securely 13
4.2.7 Control third-party and reused components 14
4.2.8 Manage exceptions, deviations and residual risk 14
4.2.9 Retain security evidence 14
4.2.10 Ensure role-based training and competence 14
4.2.11 Operate securely after release 14
4.2.12 Apply additional controls for cloud, SaaS, AI and agentic development 14
4.3 Intellectual Property Rights (IPR) 14

5 Secure Development Lifecycle Requirements for Agentic Development 15
5.1 Requirements gathering 15
5.1.1 Security Requirements 15
5.1.2 AI/Agent Risk Assessment [AGENTIC] 15
5.2 Design and architecture 15

5.2.1 Security Architecture Principles 15
5.2.2 Threat Modelling 16
5.2.3 Principle of Least Privilege for Agent APIs [AGENTIC] 16
5.3 Implementation 17
5.3.1 Use of the company GitHub tenant 17
5.3.2 Mandatory Automated Security Gates 17
5.3.3 Prompt Injection Mitigation [AGENTIC] 18
5.3.4 Secure Coding Standards 18
5.3.5 Secrets Management 18
5.4 Testing and verification 20
5.4.1 Security Testing Requirements 20
5.4.2 AI-Generated Test Suite Validation [AGENTIC] 20
5.5 Deployment and release 20
5.5.1 Cryptographic Signing of Artifacts 20
5.5.2 Production Deployment Controls 21

6 Security standards catalogue 22

7 Continuous Improvement and Exceptions 23
7.1 Security Exception Process 23
7.2 Training and competency requirements 23
7.3 Metrics and KPIs for Security Programme Health 24
7.4 Training and Competency Requirements 24
7.5 Document review and continuous improvement cycle 24

8 Key Monitoring Controls 25

9 Exceptions, deviations and risk acceptance 25

10 Glossary of terms 26

11 Alignment statement 26

12 Document information 27
12.1 Governing data 27

1 Approvals, reviews and change history 27

1 Purpose, Objectives, Scope and Applicability

1.1 Purpose and objectives

The purpose of the Secure Software Development (SSD) Process is to establish a single, company-wide and technology-agnostic approach for designing, building, testing, releasing, operating and maintaining software securely within Vivicta. It consolidates requirements from the Software Development Rules and Secure Development Lifecycle (SDLC) Rule into practical responsibilities, deliverables, controls and decision gates that project and product teams can apply consistently.

The process embeds security-by-design and privacy-by-design throughout the full software lifecycle, from project initiation through operational maintenance. It ensures that security requirements, threat modelling, secure architecture, implementation standards, security testing, component risk assessment and evidence retention are planned, performed and documented as part of normal delivery activities.

The SSD Process also defines mandatory minimum-security requirements for all software development activities conducted by, or on behalf of, Vivicta. These requirements apply to bespoke development, managed services, automation, AI-assisted development and agentic development pipelines, so that delivered software is secure by design, auditable and aligned with applicable legal, regulatory and customer obligations.

A specific objective of this document is to address the transition from exclusively human-authored code to AI-assisted and agentic software development, where autonomous or semi-autonomous AI agents may generate, test, commit or prepare code for deployment. Because these delivery models introduce additional security, governance and accountability risks, the SSD Process defines the controls and human oversight needed to manage them safely.

The main objectives of the SSD Process are to:

- Establish one unified SSD process that applies across Vivicta regardless of technology stack, delivery model, methodology, platform, or development approach.
- Define clear responsibilities, decision points and expected evidence for all roles involved in secure software delivery, including development, testing, analysis, design, product ownership, project management, consulting, delivery and service operations.
- Embed security-by-design and privacy-by-design into each lifecycle phase by requiring security requirements, threat modelling, secure architecture, secure implementation standards, validation activities, vulnerability management and release readiness controls.
- Create traceable and auditable security evidence that supports internal review, customer assurance, release decisions, exception handling and external audit activities.
- Support compliance with Vivicta security baseline requirements, ISO 27001/ISO 27002 expectations and recognised industry guidance, including OWASP SAMM, OWASP ASVS, OWASP Top 10 and secure practices for AI-assisted and agentic development.

1.2 Scope and applicability

This SSD Process applies to all software development and significant software change activities performed by, or on behalf of, Vivicta. It covers internal solutions, customer delivery software, managed services, automation, integrations, infrastructure-as-code, cloud configuration, data pipelines, AI-enabled software, and SaaS or platform configuration where Vivicta controls the configuration, delivery outcome, or operational risk.

The process is technology-agnostic and applies across delivery models, including traditional human-authored development, hybrid AI-assisted development, and agentic development pipelines. Teams must select implementation standards, controls and evidence proportionate to the system type, risk level, data classification, deployment environment, customer commitments and applicable legal or regulatory obligations.

Where a customer, contract or regulation requires more specific or stricter secure development practices, those requirements must be followed in addition to this process. Where such requirements are absent, incomplete or weaker than Vivicta requirements, this SSD Process defines the minimum Vivicta baseline unless a documented and approved deviation exists.

In practical terms, this process applies to the following organisational areas, delivery models, assets and environments:

- All Vivicta business units, delivery functions and service teams involved in designing, developing, configuring, integrating, testing, releasing, operating or maintaining software.
- All delivery pipelines and development approaches, including traditional SDLC, agile delivery, DevOps, DevSecOps, low-code/no-code automation, AI-assisted development and agentic development.
- All relevant software and configuration assets, including client-facing applications, internal tools, scripts, APIs, integrations, middleware, infrastructure-as-code, CI/CD workflows, cloud configuration, data pipelines and AI model or agent configuration files.
- All third parties, contractors, subcontractors, suppliers and offshore partners who develop, configure, integrate, test or operate software for, or on behalf of, Vivicta.
- All development, test, staging, production and production-equivalent environments, whether hosted on-premises, in public cloud, in private cloud, or in hybrid environments.

1.3 Exclusions

The following activities are outside the scope of this process unless the project, customer, contract, risk assessment or Security Plan explicitly brings them into scope:

- Pure configuration changes to commercial off-the-shelf software where Vivicta does not create, modify or integrate custom code, scripts, APIs, workflows, data processing logic or security-relevant configuration.
- Research, experimentation and proof-of-concept work performed in fully isolated environments with no production connectivity, no customer or personal data, and no path to production release without subsequent SSD classification.

2 Governance, Segregation of Duties, Roles and Responsibilities

2.1 Governance

Governance for this process and the related guidelines is defined in the document information section.

Effective secure development requires clearly defined ownership. The following roles and responsibilities apply across all Vivicta development programmes.

2.2 Roles and responsibilities

2.2.1 Standard SDLC roles

Role	Definition and responsibilities
CISO	Owns this policy; approves exceptions; escalation point for critical security defects
Security Champion	Embedded security advocate within each development team. Must be certified or credentialed (e.g. CSSLP, CEH). Responsible for threat modelling participation, security gate sign-off, and SAST/DAST triage.
Developer	Implements software, automation, configuration, or code. Must follow agreed secure coding/configuration standards, protect secrets, use approved repositories, participate in reviews, fix vulnerabilities, and report risks in the standard risk process.
Tester / QA Engineer	Plans and executes security-relevant tests, validates acceptance criteria, verifies remediation, records evidence, and confirms security testing is embedded in the test strategy.
Business Analyst	Elicits and documents security, privacy, customer, functional and non-functional requirements; ensures requirements are traceable to design, implementation, and tests.
Solution Designer / Architect / SDLC Designer	Owns solution design and security design evidence; applies secure design principles, threat modelling, risk assessment, control selection, architecture decisions, and SAR submissions where required.
DevSecOps Engineer	Operates and maintains the CI/CD pipeline security toolchain. Owns SAST, DAST, SCA tool configurations and break-build thresholds.
Product Owner	Owns product backlog prioritization; ensures security backlog items, vulnerability remediation, and required security evidence are planned and prioritized before release.
Project Manager	Plans and tracks SSD activities, dependencies, milestones, training completion, evidence collection, risk decisions, and release gate readiness.
Customer Consultant	Identifies customer security expectations, contractual obligations, service constraints, and acceptance needs; ensures customer-specific requirements are reflected in delivery artifacts.
Delivery Executive	Provides delivery-level accountability for applying SSD across engagements, resolving escalations, and ensuring sufficient capacity and governance support.
Service Delivery Manager	Ensures operational security controls, monitoring, patching, vulnerability management, incident readiness, continuity, and service evidence after go-live.
CISO / Security & Compliance	Defines and maintains security rules, principles, baseline expectations, and escalation paths; approves or reviews high-risk deviations as applicable.
Security Governance Office (SGO)	Provides security architecture/design guidance, review expectations, and support for security architecture review activities.
Data Protection Officer (DPO)	Provides privacy guidance where personal data, data protection requirements, record management, or privacy-by-design issues are involved.
SDLC Owner	Business accountable person for an SDLC; ensures role-based training, lifecycle controls, and ongoing adequacy of the lifecycle.
SDLC Operator	Operational user of an SDLC system; applies the defined lifecycle process and records evidence as required.

2.2.2 Agentic development roles

The following roles are mandatory for any project utilising AI agents in the development pipeline. These roles must be formally assigned, documented, and recorded in the project security plan before any agentic pipeline goes live.

Role	Responsibilities
AI Tooling Governance Board (ATGB)	Cross-functional committee (minimum: CISO, Head of Engineering, Legal, DPO). Must approve and periodically audit all AI coding tools, LLM providers, and agent orchestration platforms before deployment. Meets quarterly minimum and on any material toolchain change.
Agent Orchestrator / Config Owner	Named individual (not a team) accountable for the configuration, prompt design, tool access scope, and operational behaviour of each deployed AI agent. Must maintain a versioned Agent Configuration Record.
Human-in-the-Loop (HITL) Reviewer	Named engineer who performs mandatory review at defined HITL checkpoints. Must be independent — an agent that generates code cannot also be designated as the HITL reviewer for that code. Must have relevant security clearance for the data classification of the project.
AI Security Auditor	Performs periodic red-team assessments of agent pipelines, including prompt injection testing and adversarial input validation.

2.3 Mandatory Segregation of Duties

The following segregation of duties (SoD) requirements are mandatory and must be enforced through technical controls rather than policy statements alone:

- An AI agent that generates code must not also be the entity that approves, merges, or deploys that code to production. Technical controls (branch protection rules, required human approvers) must enforce this.
- The Agent Orchestrator / Config Owner must not also serve as the HITL Reviewer for the same agent's output.
- Security Champions must report to the CISO's function for security matters, not solely to the Product Owner, to prevent commercial pressure overriding security gates.
- Access to production secrets and credentials must be restricted to deployment automation pipelines only; no individual developer or AI agent may hold persistent production credentials.

2.4 RACI matrix

R = Responsible, A = Accountable, C = Consulted, I = Informed. Accountable is assigned once per activity wherever possible.

Activity	Dev	Tester	BA	Architect	PO	PM	CISO	SGO	DPO	ATGB	Agent Owner	HITL	AI Auditor
Classify initiative and confirm SSD applicability	C	I	R	C	A	R	C	C	C	I	C	I	I
Assign SSD and agentic development roles	I	I	C	C	A	R	C	C	I	C	R	R	I
Capture security, privacy, customer and regulatory requirements	C	C	R	C	A	R	C	C	C	C	C	C	I
Define secure architecture, data flows and threat model	C	C	C	A/R	C	I	C	C	C	I	C	I	C
Approve AI tools, LLM providers and agent orchestration platforms	I	I	I	C	I	I	C	C	C	A/R	R	I	C
Configure and maintain AI agent behaviour, access scope and records	C	I	I	C	I	I	C	I	I	C	A/R	I	C
Implement code/configuration securely	A/R	I	I	C	C	I	I	I	I	I	C	I	I
Review code, agent output and repository controls	R	C	I	C	C	I	I	I	I	I	C	A/R	C
Plan and execute security testing, including agent pipeline testing	C	A/R	C	C	C	R	C	C	I	I	C	C	R
Approve deviations, residual risk and SoD exceptions	C	C	C	C	A	R	A	C	C	C	C	I	C
Release readiness and secure deployment decision	C	C	I	C	A	R	C	C	I	I	C	C	I
Maintain SSD process, guidance, training and governance evidence	I	I	I	C	C	C	A	R	C	C	C	I	C

3 Architecture of an Agentic Development Pipeline

This section defines mandatory architectural requirements for any Vivicta development pipeline in which AI agents generate, modify, test, or commit code. These requirements apply regardless of the underlying LLM provider or orchestration framework.

3.1 Mandatory Secure Sandboxing Requirements

All AI agents performing code generation or execution must operate within a dedicated, isolated execution environment. The following controls are mandatory:

Control	Requirement	Enforcement Method
Network Isolation	Agent execution environments must have no direct internet egress except to explicitly allowlisted endpoints (e.g., approved package registries, LLM API endpoints). All other outbound traffic must be blocked at the network layer.	VPC/subnet rules; egress firewall
Filesystem Isolation	Agents must operate within a read-only filesystem with write access limited to explicitly defined, isolated working directories. They must not have access to host filesystems, other project workspaces, or secrets stores.	Container (Docker/OCI) with read-only root; tmpfs for working dirs
Credential Isolation	Agents must not hold persistent credentials. All tool access must be via short-lived, scoped tokens granted at runtime by an identity provider (e.g., Vault dynamic secrets, OIDC federation).	HashiCorp Vault, AWS IAM Roles Anywhere, or equivalent
Resource Limits	CPU, memory, and execution time limits must be set on all agent containers to prevent resource exhaustion and runaway loops.	Kubernetes ResourceQuotas; container runtime limits
Audit Logging	All agent actions — including every tool call, file write, API call, and LLM prompt/response — must be logged to an immutable audit trail. Logs must be retained for a minimum of 12 months.	Centralised SIEM (e.g., Splunk, Microsoft Sentinel); write-once storage
Image Integrity	Agent container images must be signed and their signatures verified at deployment time. Use of unverified or unsigned images is prohibited.	Sigstore/Cosign; OPA/Gatekeeper admission controller

3.2 Deterministic Guardrails: Human-in-the-Loop (HITL) Triggers

HITL review is mandatory before an agent's output may proceed to the next pipeline stage in all of the following high-risk scenarios. These triggers must be implemented as hard gates in the CI/CD pipeline — not as advisory warnings.

Trigger Category	Specific Condition	Required HITL Action
Identity & Authentication	Agent generates or modifies any code related to authentication, authorisation, session management, or identity federation (OAuth, SAML, OIDC).	Named Security Champion reviews and provides explicit approval via signed commit or pull request approval.
Cryptography	Agent generates or modifies any cryptographic implementation, key management code, or certificate handling logic.	Security Champion with cryptography competency (documented) must review. Automated SAST crypto-check must pass first.
External API Integration	Agent writes code that calls or is called by an external API, particularly one handling payments, personal data, or regulated information.	Product Owner approves business justification; Security Champion reviews integration security (auth, data handling, error paths).
Database Schema Changes	Agent proposes changes to production database schemas, migration scripts, or ORM models.	Database Administrator and Security Champion must jointly review.
Infrastructure-as-Code (IaC)	Agent generates or modifies IaC (Terraform, Bicep, CloudFormation) affecting network security groups, IAM policies, or storage access controls.	DevSecOps Engineer and Security Champion review. Automated policy-as-code scan (e.g., Checkov, OPA) must pass first.
Secrets & Configuration	Agent generates any file containing credential references, environment variable definitions, or configuration that could affect secrets handling.	Security Champion verifies no hardcoded secrets and that secrets retrieval follows the Secrets Management Standard (Section 5.3.4).
Production Deployment	Any deployment to a production or production-equivalent environment.	HITL approval from both the Product Owner and Security Champion. No agent may autonomously approve or trigger a production deployment.

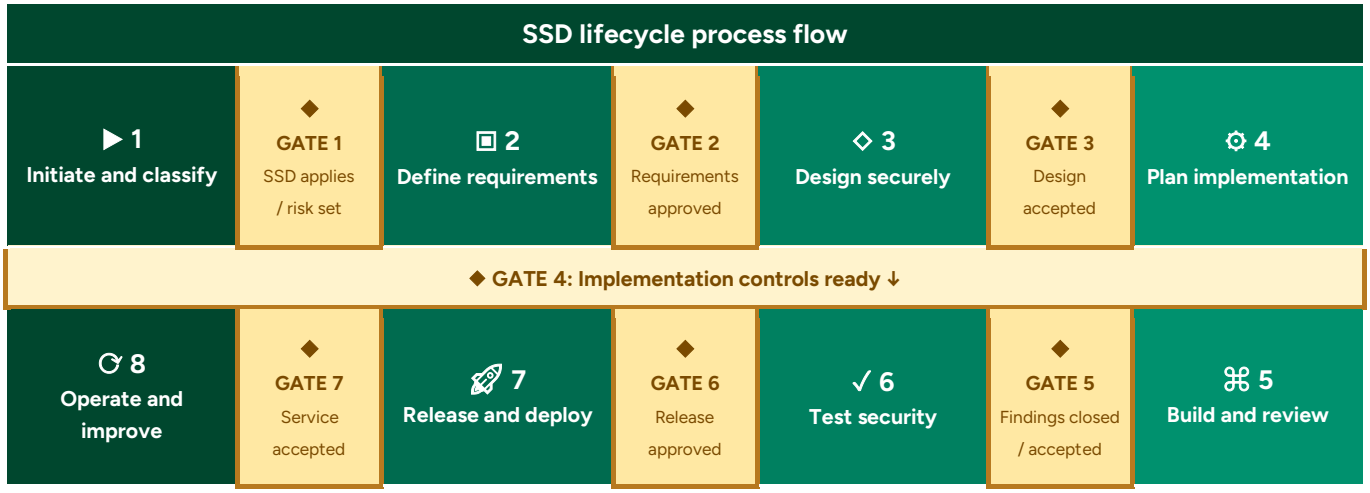
3.3 Agent Prompt & Tooling Governance

The following controls govern how agents are configured and what capabilities they are granted:

- Agent system prompts must be version-controlled in a dedicated repository with access restricted to the Agent Orchestrator / Config Owner. Changes to system prompts must follow the same change management process as code changes.
- Agents must be granted only the minimum tool access required for their specific task (principle of least privilege). Tool access must be reviewed and re-approved quarterly.
- Agent-to-agent communication (multi-agent architectures) must use authenticated, signed message passing. Agents must not accept instructions from untrusted orchestrators.
- The use of external, third-party AI agents or services within a Vivicta development pipeline requires prior approval from the ATGB and a completed Data Processing Agreement (DPA) with the provider.

4 SSD Lifecycle Process Overview, Evidence Model and Process Rules

4.1 Process overview



Phase	Purpose	Key activities	Gate / decision	Editable evidence
1. Initiate and classify	Confirm whether the initiative is in scope and determine the required SSD depth.	Classify the initiative, assign owner and roles, identify lifecycle model, data classification, customer obligations, delivery model and initial risk level.	SSD applies / does not apply; risk level assigned.	Classification record; role mapping; initial risk record; security plan placeholder.
2. Define requirements	Make security, privacy and compliance needs explicit before design and build start.	Capture customer, legal, regulatory, operational, security and privacy requirements; define acceptance criteria; make requirements traceable to design, implementation and testing.	Requirements approved and traceable.	Security requirements register; acceptance criteria; traceability matrix.
3. Design securely	Translate requirements into a secure architecture and risk-based control design.	Define architecture, data flows, trust boundaries, secure design principles, threat model, security risks, controls and residual risks; complete SAR activities where applicable.	Design and risk position accepted.	Architecture/design record; threat model; risk log; control mapping; SAR evidence if required.
4. Plan implementation	Prepare the secure delivery setup before coding or configuration work proceeds.	Select secure coding/configuration standards, define repository and branch controls, plan security tooling, dependency management, secrets handling, review model and agentic controls where applicable.	Implementation controls ready.	SSD directives; secure standard selection; repository settings; tool plan; agent configuration record if applicable.
5. Build and review	Create code, configuration or automation using controlled and reviewable delivery practices.	Implement securely, protect secrets, manage dependencies, review code and configuration, validate agent output through HITL checkpoints, and keep build and repository controls active.	Code/configuration review completed.	Pull request evidence; review approvals; SAST/SCA results; dependency inventory/SBOM; HITL records if applicable.
6. Test security	Verify that security requirements and controls work as intended.	Execute risk-proportionate security testing, including functional security tests, SAST, SCA, DAST, secrets scanning, IaC/configuration checks,	Findings remediated or accepted.	Security test plan; scan reports; defect log; remediation evidence; residual risk approval.

		API tests, penetration testing and agent pipeline tests where required.		
7. Release and deploy	Make a controlled release decision based on evidence, risk and deployment readiness.	Confirm release criteria, approve deviations or residual risks, verify deployment controls, preserve segregation of duties, confirm rollback and operational ownership.	Release approved / deferred.	Release checklist; deployment approval; risk acceptance; deployment record; operational handover.
8. Operate and improve	Maintain security after go-live and feed lessons learned back into the process.	Operate monitoring and logging, manage vulnerabilities and patches, respond to incidents, review controls, update evidence, and reassess risks during significant changes.	Service accepted for operation; improvements tracked.	Monitoring evidence; vulnerability records; incident readiness evidence; post-implementation review; improvement backlog.

4.2 SSD process rules

4.2.1 Plan security from the start

Security and privacy must be considered from the start of every software development, system configuration, integration, automation or acquisition activity. Teams must identify the applicable security context early, including data classification, intended use, customer commitments, legal and regulatory expectations, delivery model, technology stack, cloud or SaaS dependencies and any AI or agentic development involvement.

4.2.2 Define and trace security requirements

Security, privacy, customer, operational and compliance requirements must be captured before design and build activities proceed. Requirements must be explicit, testable and traceable through design, implementation, verification, release and operation so that each security expectation can be linked to evidence and a release decision.

4.2.3 Analyse threats, risks and design controls

All in-scope systems must undergo threat modelling or security risk analysis proportionate to complexity, exposure and data classification. The analysis must identify assets, data flows, trust boundaries, misuse cases, likely threats, vulnerabilities, impact, likelihood, required controls, risk owners and residual risks. Agentic development pipelines must also consider prompt manipulation, unsafe tool use, agent autonomy, data leakage and failure modes.

4.2.4 Apply secure design principles

Designs must apply security-by-design, privacy-by-design, defence in depth, secure-by-default configuration, least privilege, default deny, fail secure, complete mediation and manageable operations. Any principle that cannot be fully applied must be documented with rationale, compensating controls, residual risk and approval.

4.2.5 Select and follow implementation standards

Teams must select secure implementation standards appropriate to the technology, architecture, delivery model and risk level. Relevant sources may include Vivicta secure coding and configuration standards, OWASP ASVS, OWASP Top 10, OWASP SAMM, SEI CERT, cloud or platform benchmarks, customer-specific requirements and approved standards for AI-assisted or agentic development.

4.2.6 Build, review and validate securely

Security validation must be embedded into normal development activities. Teams must apply appropriate automated and manual controls, such as code review, configuration review, SAST, SCA, DAST, secrets detection, infrastructure-as-code scanning, API testing, penetration testing and HITL review of agent-generated output where applicable.

4.2.7 Control third-party and reused components

Reusable components, open-source libraries, templates, APIs, SaaS features, infrastructure modules, AI services and agent tooling must be registered or traceable, assessed for vulnerability, license, supplier and data-risk exposure, and maintained through an update, remediation or replacement process.

4.2.8 Manage exceptions, deviations and residual risk

Any bypassed control, skipped activity, unsupported design pattern, unresolved vulnerability, non-compliant component, missing test, failed gate or unmet standard must be documented before release. The record must include the risk, owner, rationale, compensating controls, approval authority, review date and expiry or remediation plan.

4.2.9 Retain security evidence

Security requirements, design decisions, threat models, risk assessments, approvals, reviews, scan results, test evidence, exceptions, release sign-offs and operational security records must be retained in the project or service evidence repository. Evidence must be sufficient to support internal review, customer assurance, audit and future maintenance.

4.2.10 Ensure role-based training and competence

Personnel who design, develop, configure, integrate, test, release, operate or govern software must have up-to-date secure development training appropriate to their role, technology and delivery model. Teams using AI-assisted or agentic development must also understand the related governance, review and segregation-of-duties controls.

4.2.11 Operate securely after release

Security responsibilities continue after go-live. Teams must maintain logging, monitoring, vulnerability and patch management, backup and restore readiness, incident response alignment, secure deletion and records management, periodic control review and risk reassessment for significant changes.

4.2.12 Apply additional controls for cloud, SaaS, AI and agentic development

Where software uses public cloud, SaaS, AI agents, generative AI or AI-enabled development tools, teams must address data classification, residency, access management, supplier risk, logging, model or agent permissions, prompt and tooling governance, secure deletion, customer obligations and human oversight requirements before use in delivery or production.

4.3 Intellectual Property Rights (IPR)

Vivicta shall identify and document the Intellectual Property Rights (IPR) model applicable to each customer engagement before development commences, including any agreed licenses or transfers of ownership. As a general principle, Vivicta must retain ownership of software, code, frameworks, reusable components, and other deliverables developed by Vivicta, while customers are granted the rights necessary to use the delivery for their internal business purposes. Any customer-specific ownership transfer of software or source code must be expressly approved, clearly defined in the relevant customer agreement, and limited to assets that are separable from Vivicta's background IP and reusable components. Development teams must maintain traceability of all customer-owned deliverables and ensure that code, components, or materials subject to transferred ownership are not reused in other customer projects to avoid infringement of customer IPR. Third-party software, open-source components, and licensed technology must remain subject to their respective license terms and must never be included in any transfer of ownership unless expressly permitted by the applicable rights holder.

5 Secure Development Lifecycle Requirements for Agentic Development

The following requirements apply across the software development lifecycle. Controls marked with [AGENTIC] apply specifically to, or are strengthened for, agentic development pipelines.

5.1 Requirements gathering

5.1.1 Security Requirements

Security acceptance criteria must be defined as explicit, testable requirements within every user story or functional specification. Acceptance criteria must address: authentication and authorisation requirements; data classification and handling obligations; applicable regulatory constraints; expected attack surface and known threat vectors.

5.1.2 AI/Agent Risk Assessment [AGENTIC]

Before any agentic development pipeline is instantiated, the project team must complete an AI Risk Assessment covering:

- Threat modelling of agent prompts: what adversarial inputs could manipulate the agent's behaviour? Document mitigations.
- Tool access audit: for every tool the agent may invoke (code execution, file system, APIs), document the data it can access and the actions it can take.
- LLM provider risk: assess data residency, training data policies, and confidentiality obligations of the chosen LLM provider against project data classification.
- Failure mode analysis: what happens if the LLM returns hallucinated code, incorrect logic, or adversarially influenced output? Document detection and rollback procedures.

The AI Risk Assessment must be reviewed and approved by the Security Champion and the ATGB before pipeline activation.

5.2 Design and architecture

5.2.1 Security Architecture Principles

All software design must apply the following principles: Secure by Default (secure configuration must be the out-of-the-box state); Defence in Depth (multiple independent security controls); Least Privilege (every component operates with the minimum access needed); Fail Secure (failures must not leave the system in an insecure state); Complete Mediation (every access attempt to a protected resource must be authorised).

5.2.2 Threat Modelling

Threat modelling using STRIDE or an equivalent methodology is mandatory for all components rated as High or Critical data classification, and for all agentic pipeline components regardless of data classification. Threat models must be documented, version-controlled, and updated at each major design revision.

5.2.3 Principle of Least Privilege for Agent APIs [AGENTIC]

Data flow diagrams mapping the movement of data between LLMs, code repositories, external APIs, and databases must be produced and maintained for all agentic pipelines. These diagrams must identify: every trust boundary crossed; the classification of data at each point; the authentication mechanism at each interface; and the logging coverage at each data flow.

- Agent API credentials must use the minimum OAuth scopes or IAM permissions required. Wildcard permissions (e.g., *) are prohibited.
- Agents must not have write access to the main/trunk branch of any code repository. All agent commits must be to feature branches subject to pull request review.
- LLM API calls must not include production data, personal data, or confidential business data in prompts unless the LLM provider has been approved for that data classification by the DPO.

5.3 Implementation

5.3.1 Use of the company GitHub tenant

All Vivicta software development repositories, including repositories used by AI-assisted or agentic development pipelines, must be hosted in the approved company GitHub tenant unless an approved exception exists. Personal GitHub accounts, private non-company repositories, unmanaged organization accounts, and customer or supplier repositories must not be used for Vivicta-controlled source code, configuration, CI/CD workflows, infrastructure-as-code, agent prompts, or security evidence.

Access to the company GitHub tenant must be provisioned through approved identity and access management processes, using company-managed identities, role-based access, multi-factor authentication, and least-privilege repository permissions. Repository ownership, branch protection rules, required pull request reviews, mandatory security checks, secrets scanning, dependency scanning, and audit logging must be configured according to Vivicta secure development requirements.

AI agents and automation tools must interact only with repositories in the company GitHub tenant and must use short-lived, scoped credentials or approved GitHub App/OIDC-based access. AI agents must not use personal access tokens, unmanaged service accounts, or direct write access to protected branches. Any use of GitHub outside the company tenant requires prior documented approval, risk assessment, and compensating controls.

5.3.2 Mandatory Automated Security Gates

The CI/CD pipeline must implement the following automated security checks as hard break-build gates. Pipelines that fail any of these checks must not proceed to the next stage without explicit exception approval from the CISO.

Gate	Tool Category (Examples)	Break Condition	Applies To
Static Application Security Testing (SAST)	Semgrep, SonarQube, Checkmarx, Snyk Code	Any Critical or High severity finding not acknowledged with an approved risk acceptance	All code changes
Software Composition Analysis (SCA)	Snyk Open Source, OWASP Dependency-Check, Black Duck	Critical CVE in any direct or transitive dependency; dependency with no upstream maintenance	All code changes
Dynamic Application Security Testing (DAST)	OWASP ZAP, Burp Suite Enterprise, StackHawk	Critical or High finding; any unauthenticated access to protected endpoints	Pre-production environment; any API change
Secrets Scanning	GitGuardian, Trufflehog, GitHub Advanced Security	Any secret, credential, or private key detected in code or commit history	All commits; pre-commit hook mandatory
Infrastructure-as-Code (IaC) Scanning	Checkov, tfsec, KICS	Critical misconfiguration (open security groups, public storage, unencrypted data at rest)	All IaC changes

Container Image Scanning	Trivy, Anchore, Aqua Security	Critical CVE in base image or installed packages	All container image builds
AI-Generated Code Analysis [AGENTIC]	SAST (as above) + LLM-specific rules (custom Semgrep rulesets for prompt injection patterns)	Same as SAST break condition; additionally: any code that constructs SQL/shell/OS commands from agent-controlled input	All agent-generated code changes

5.3.3 Prompt Injection Mitigation [AGENTIC]

Prompt injection — the manipulation of an AI agent through adversarial input embedded in data the agent processes — is a primary threat vector in agentic development pipelines. The following controls are mandatory:

- Strict input/output separation: agents must treat all externally sourced data (code comments, documentation, issue tracker content, test data) as untrusted and must not allow such data to modify agent instructions or tool call parameters.
- Structured prompting: use structured formats (JSON, XML) for tool inputs and outputs rather than freeform text to reduce injection surface.
- Output validation: all agent-generated outputs that are used as inputs to subsequent pipeline stages must be validated against a defined schema or whitelist before use.
- Content security policies for LLM prompts: maintain an approved prompt template library. Ad-hoc prompt construction by pipeline code is prohibited without ATGB approval.
- Regular red-team testing: the AI Security Auditor must conduct prompt injection testing against all production agentic pipelines at least quarterly.

5.3.4 Secure Coding Standards

All developers (human and AI agents via their configurations) must comply with Vivicta's Secure Coding Standard, which incorporates OWASP Top 10, SANS Top 25, and relevant language-specific guidance. Specific mandatory practices include:

- Parameterised queries / prepared statements for all database interactions. String concatenation for SQL construction is prohibited.
- Output encoding appropriate to context (HTML, URL, JavaScript, CSS) for all data rendered in browser contexts.
- Centralised authentication and session management libraries must be used. Custom authentication implementations require CISO approval.
- Error handling must not expose stack traces, internal paths, or sensitive data to end users or logs accessible by non-privileged parties.

5.3.5 Secrets Management

The following rules governing secrets management are mandatory and must be enforced by both technical controls and pre-commit hooks:

- PROHIBITION: No credential, API key, certificate, password, or secret of any kind may be hardcoded in source code, configuration files, IaC templates, container images, or Dockerfiles. This prohibition applies to AI-generated code equally.
- MANDATORY: All secrets must be retrieved dynamically at runtime from an approved secrets management system. Vivicta's approved platforms are: HashiCorp Vault (on-premises/self-hosted), Azure Key Vault (cloud-hosted), and AWS Secrets Manager (cloud-hosted). Alternatives require ATGB approval.
- MANDATORY [AGENTIC]: AI agents must not be provided with secrets directly. Agents that require access to APIs or services must receive short-lived, scoped tokens via a secrets injection mechanism at the point of use, which expire upon task completion.
- MANDATORY: All secrets used in CI/CD pipelines must be stored in the pipeline's native secrets store (e.g., GitHub Actions Secrets, GitLab CI/CD Variables set to Masked and Protected) and must never appear in pipeline logs.
- Secret rotation must occur at the following cadences: API keys: 90 days maximum; Database credentials: 90 days maximum; Service account tokens: 60 days maximum; Human user credentials: per the Access Management Policy.

5.4 Testing and verification

5.4.1 Security Testing Requirements

The following security testing activities are mandatory before any release candidate may be promoted to production:

- Penetration testing: for new applications or major feature releases affecting the attack surface, an independent penetration test (internal red team or approved third party) must be conducted. Scope, findings, and remediation status must be documented.
- Security regression testing: the automated security gate suite must be executed in full on every pull request and on every build of a release candidate.
- Privacy impact assessment: any software processing personal data must have a completed and approved Data Protection Impact Assessment (DPIA) before go-live.

5.4.2 AI-Generated Test Suite Validation [AGENTIC]

AI agents generating test code present a specific risk: an agent may produce tests that pass against flawed code it also generated, either through overfitting or through optimising for test passage rather than correct behaviour. The following controls are mandatory:

- Test independence: AI-generated tests must be reviewed and validated by a human engineer independent of the agent that generated the code under test. The reviewer must verify that tests exercise meaningful negative paths and failure conditions, not only the happy path.
- Mutation testing: for Critical and High-classification components, mutation testing tools (e.g., PITest for Java, Mutmut for Python) must be used to verify that the test suite genuinely detects code defects. A minimum mutation score of 70% is required.
- Code coverage is necessary but insufficient: 100% line coverage does not validate security. HITL reviewers must assess test quality, not only coverage metrics.
- Security-specific test cases: HITL reviewers must verify that security-relevant test cases (e.g., authentication bypass attempts, injection payloads, boundary conditions) are present and correctly assert rejection of malicious input.

5.5 Deployment and release

5.5.1 Cryptographic Signing of Artifacts

All software artifacts promoted to production must be cryptographically signed to ensure integrity and establish non-repudiation:

- Container images must be signed using Sigstore/Cosign or an equivalent framework. Unsigned images must be rejected by the container orchestration platform's admission controller.
- Build artifacts (JARs, NPM packages, binaries) must be signed and their signatures published alongside the artifact. The signing key must be managed in the approved secrets management platform.
- Software Bill of Materials (SBOM) in SPDX or CycloneDX format must be generated and signed for every production release and retained for the duration of the software's support lifecycle.

- Signatures must be verified automatically as part of the deployment pipeline before any artifact is executed in a production environment.

5.5.2 Production Deployment Controls

- ABSOLUTE PROHIBITION: No AI agent may autonomously approve, merge, or trigger a deployment to any production or production-equivalent environment. This control must be enforced by technical pipeline configuration (required human approvers, protected branch rules), not solely by policy.
- All production deployments must follow the Change Management Process defined in VIV-SEC-POL-008. Emergency changes are subject to accelerated but not eliminated HITL review.
- Deployment pipelines must implement automated rollback capabilities. Rollback procedures must be tested as part of each release cycle.
- Post-deployment verification: automated smoke tests and security health checks must run immediately following every production deployment. A failed post-deployment check must trigger automatic rollback.

6 Security standards catalogue

The following catalogue is technology agnostic. Teams must select the standards relevant to their solution type and record the selection in the SSD Directives or equivalent project security plan.

Standard / Source	SSD usage	Applicability
OWASP SAMM	Use as the maturity and governance framework for improving the SSD program across Governance, Design, Implementation, Verification, and Operations.	Company/process maturity, gap assessment, roadmap, continuous improvement.
OWASP ASVS	Use as detailed application/API security requirements and verification criteria.	Web applications, web APIs, web services, high-assurance application security requirements.
OWASP Top 10	Use as awareness baseline and minimum web application risk checklist; not sufficient by itself for high-risk systems.	Developer education, initial risk review, baseline web security checks.
SEI CERT standards	Use where language-specific secure coding standards are available and relevant.	C, C++, Java, Android, Perl or other applicable technology-specific secure coding.
Cloud/provider security benchmarks	Use cloud platform security standards and hardening guidance relevant to the selected platform and customer/regulatory obligations.	Public cloud, hybrid cloud, container platforms, managed services.
Vivicta security principles and baseline rules	Use as mandatory internal baseline for architecture, SDLC, cyber controls, privacy, cloud, AI, logging, cryptography, records, and data protection.	All Vivicta software development and significant change activities.

7 Continuous Improvement and Exceptions

7.1 Security Exception Process

Exceptions to any control defined in this document may be granted only under the following conditions and through the following process. Informal exceptions — verbal agreements, workarounds without documentation — are prohibited.

Step	Action	Owner	Timeframe
1	Exception request submitted using the Security Exception Request Form (VIV-SEC-FORM-003), including: specific control being excepted; business justification; risk assessment; proposed compensating controls; requested duration.	Requesting team	Before exception is needed
2	Review and risk assessment by Security Champion and CISO (or delegate).	CISO function	5 business days
3	For exceptions involving agentic pipelines or LLM tools: ATGB review and approval required.	ATGB	10 business days
4	Approved exceptions recorded in the Security Exception Register with expiry date, compensating controls, and review date. Maximum exception duration: 90 days (renewable with re-approval).	CISO function	At approval
5	Exception tracked on the project risk register and reported in quarterly security governance reporting.	Security Champion	Ongoing

7.2 Training and competency requirements

When a new vulnerability, exploit technique, or LLM attack vector is publicly disclosed or internally discovered:

- Critical vulnerabilities affecting production systems or active agentic pipelines: the CISO must be notified within 4 hours. An emergency guardrail update or temporary pipeline suspension may be enacted without full change management review (but must be retrospectively documented within 48 hours).
- High vulnerabilities: SAST/SCA rulesets must be updated within 5 business days. Existing codebases must be re-scanned. Remediation plan must be documented within 10 business days.
- Agent guardrail updates: any update to agent system prompts, tool access configurations, or sandbox rules triggered by a security event must be recorded in the Agent Configuration Record, reviewed by the ATGB, and reflected in an update to this document's next revision.
- OWASP LLM Top 10 updates: upon publication of a new version of OWASP LLM Top 10, the CISO must conduct a gap assessment within 60 days and update this document accordingly.

7.3 Metrics and KPIs for Security Programme Health

The following metrics must be measured and reported monthly to the CISO and quarterly to executive leadership:

Metric	Target	Escalation Threshold
Mean Time to Remediate (MTTR) — Critical vulnerabilities	< 24 hours	Any breach triggers CISO notification
MTTR — High vulnerabilities	< 7 days	Breach triggers Security Champion escalation
% of CI/CD pipelines with all mandatory security gates enabled	100%	Any pipeline below 100% is a critical finding
% of agentic pipeline HITL triggers with documented review records	100%	Below 100% is an audit finding
SAST/SCA findings per 1,000 lines of agent-generated code vs. human-authored code	Agent ≤ Human baseline	Sustained increase triggers pipeline review
Security exceptions open beyond expiry date	0	Any expired exception is escalated to CISO
% of releases with completed SBOM	100%	Any missing SBOM blocks release promotion

7.4 Training and Competency Requirements

The following training requirements are mandatory:

- All software developers: annual secure coding training. Completion is tracked and non-completion is a performance issue.
- Security Champions: certified training in application security (CSSLP, GWEB, or equivalent) within 12 months of appointment.
- Agent Orchestrator / Config Owners: mandatory training in LLM security risks, prompt injection, and agentic system governance before pipeline activation.
- HITL Reviewers: must complete AI code review training covering: how to evaluate agent-generated code; common LLM failure modes; prompt injection indicators in generated code.
- ATGB members: annual briefing on developments in AI security, regulatory landscape, and emerging agentic attack vectors.

7.5 Document review and continuous improvement cycle

This document must be reviewed annually as a minimum. The review process must:

- Assess all security incidents in the preceding 12 months for lessons that should be incorporated into these guidelines.
- Evaluate changes to the regulatory and standards landscape (ISO updates, OWASP updates, EU AI Act guidance).
- Review the effectiveness of controls based on security metrics from Section 7.3.
- Incorporate feedback from internal and external audits.

The updated document must be approved by the CISO and the ATGB before publication and must be communicated to all in-scope personnel within 10 business days of publication.

8 Key Monitoring Controls

Key Monitoring Control	Responsible role	Scope
SSD Process is defined, approved, published, and accessible to relevant stakeholders.	SSD Process Owner	Vivicta
SSD applicability classification is completed for each software/system development initiative.	Product Owner / Project Manager	Vivicta
Roles are assigned for SSD execution and recorded in project/service documentation.	Product Owner / Project Manager	Vivicta
Security requirements are captured early, risk-based, and traceable.	Business Analyst / Solution Designer	Vivicta
Threat modeling or security risk analysis is performed and documented proportionate to risk.	Solution Designer / Architect	Vivicta
Secure design principles are applied and documented in the system design.	Solution Designer / Architect	Vivicta
Secure implementation/coding standards are selected and recorded in SSD Directives or equivalent.	Solution Designer / Developer	Vivicta
Security testing is planned, executed, and evidenced before release.	Tester / QA Engineer	Vivicta
Reused components are registered/tracked and assessed for security and trustworthiness.	Developer / Solution Designer	Vivicta
Security findings, deviations, and residual risks are logged, owned, and approved before release.	Product Owner / Project Manager	Vivicta
Security evidence is retained and available for internal review or audit.	Project Manager / Service Delivery Manager	Vivicta
Role-based secure development training is assigned, completed, and refreshed.	SDLC Owner / Line Manager	Vivicta
Operational monitoring, logging, patching, and vulnerability management are active after go-live.	Service Delivery Manager	Vivicta

9 Exceptions, deviations and risk acceptance

A deviation is any case where a required SSD activity, standard, test, control, evidence item, or remediation action cannot be completed as required. Deviations must be exceptional, time-bound, risk-assessed, and approved before release or continued operation.

- Each deviation must document description, affected system, reason, security impact, likelihood, impact, compensating controls, owner, approver, expiry/review date, and remediation plan.
- High-risk deviations must involve the relevant security authority and, where personal data is involved, the DPO must be consulted.
- Known vulnerabilities released into production must have an owner, risk acceptance, monitoring plan, and remediation due date.
- Repeated deviations must trigger process improvement or management escalation.

10 Glossary of terms

Term	Definition
SSD	Secure Software Development: the company process for integrating security into software delivery from initiation through operation.
SDLC	Secure Development Lifecycle: structured lifecycle integrating security considerations into each phase of system, service, or software development.
System	A set of interrelated components working together for a purpose, including technology, data, people, processes, services, facilities, and organizational structures.
Threat model	Structured analysis of how a system could be attacked, misused, or misconfigured and what controls are required.
Risk acceptance	Formal decision by an accountable role to accept residual risk after understanding impact, likelihood, and compensating controls.
Security evidence	Records proving that required security activities, decisions, tests, reviews, and approvals were performed.
SSD Directives	Project-specific document or plan defining applicable security standards, controls, testing approach, evidence, and responsibilities.
SAST	Static Application Security Testing: analysis of source or compiled code without executing the application.
DAST	Dynamic Application Security Testing: runtime security testing of an executing application or service.
SCA	Software Composition Analysis: identification and risk assessment of third-party/open-source components.
SBOM	Software Bill of Materials: inventory of software components and dependencies used in a system.

11 Alignment statement

The requirements outlined in this Process are aligned with and embedded into applicable Vivicta Rules, Processes, and Procedures. Deviations are directed to the relevant Process or Rule Owner according to established procedures to mitigate and prevent recurrence. Suspected process or rule violations are reported to the direct manager or relevant escalation point to ensure that applicable mitigating and disciplinary actions are taken.

12 Document information

12.1 Governing data

Document Owner	Tomas Faltys
Document Manager	Jozef Harbulyak
Authority reference	ISO 27001:2022 / ISO 27002:2022 control 8.25; Vivicta Software Development Rules; Vivicta SDLC Rule
Related governing documents	Software Development Rules, Secure Development Lifecycle (SDLC) Rule, Secure Engineering & Architecture Design Rule, Security Architecture Review Guide

1 Approvals, reviews and change history

Version	Date	Editor	Reviewed by	Approved by	Changes
V1.0A	2026-09-29	Manish Kumar, Jozef Harbulyak, Ingrid Lind	Process Governance Network	Tomas Faltys	First version release, consolidation of Vivicta software development and SDLC rules; added role definitions, RACI matrix, technology-agnostic SSD rules, OWASP references, and document management sections.